

Exploring D3 Implementation Challenges on Stack Overflow

Leilani Battle*
University of Washington

Danni Feng†
University of Maryland

Kelli Webber‡
University of Maryland

ABSTRACT

Visualization languages help to standardize the process of designing effective visualizations, one of the most prominent being D3. However, few researchers have analyzed at scale how users incorporate these languages into existing visualization programming processes, i.e., *implementation workflows*. In this paper, we present an analysis of the experiences of D3 users as observed through Stack Overflow, summarizing common D3 implementation workflows and challenges discussed online. Our results show how the visualization community may be limiting its understanding of users' visualization implementation challenges by ignoring the larger context in which languages such as D3 are used. Based on our findings, we suggest new research directions to enhance the user experience with visualization languages. All our data and code are available at: <https://osf.io/fup48/>.

Keywords: Web mining, visualization language evaluation

Index Terms: Human-centered computing—Visualization—Visualization design and evaluation methods

1 INTRODUCTION

Visualization languages provide great flexibility in programming and reusing visualization designs [10, 27]. Browser-based languages in particular have made it easier for a wide range of people to experiment with programming visualizations online [6, 10, 38], one of the most influential being D3 [10]. On the one hand, D3 is wildly successful: it has been starred over 101 thousand times on GitHub and has been identified on many thousands of webpages, including those of highly regarded media outlets [6]. On the other hand, D3 also has steep learning curve, and can be challenging for analysts and data enthusiasts to adopt [27, 37].

One approach to addressing this problem is to investigate the challenges D3 users face with existing visualization programming processes and toolsets, i.e., *implementation workflows*. This knowledge could be used to strengthen existing infrastructure for implementing D3 visualizations (e.g., documentation, examples, tutorials), and even lead to targeted support tools that integrate directly with these workflows. Furthermore, D3 is an important baseline for comparison with other visualization languages [39], and existing languages adopt a similar support structure (e.g., Vega [39] and Vega-Lite [38]). By investigating how to address implementation challenges for D3, similar improvements could be propagated to other languages.

However, we observe a dearth of academic corpora regarding how users interact with visualization languages such as D3. Given the thousands of Stack Overflow posts made about D3, online forums could be a rich resource regarding how D3 users implement new visualizations, and the challenges they run into along the way. Furthermore, these forums could enable us to study D3 users on a scale not yet seen in existing evaluations [24].

*e-mail: leibatt@cs.washington.edu

†e-mail: fengdn@terpmail.umd.edu

‡e-mail: kwebber1@terpmail.umd.edu

In this paper, we start to fill this gap by mining and analyzing 37,815 posts about D3 collected from Stack Overflow. To ground our analysis, we evaluate how D3's original design goals have been realized among these users to improve compatibility, debugging, and performance [10]. Through this work, we contribute:

1. A mixed methods analysis of the experiences of D3 users as observed on Stack Overflow. To the best of our knowledge, this paper is the *first* to center on online forums, which is known to be a critical resource for D3 users.
2. A collection of common implementation workflows discussed by D3 users on Stack Overflow, common challenges when integrating D3 into these workflows, and common debugging patterns for D3 programs.
3. Empirical support for several “common knowledge” assumptions made about D3 users that have yet to be rigorously evaluated, such as the frequent incorporation of non-visualization tools into D3 implementation workflows and persistent barriers to finding relevant D3 examples online.
4. Based on our findings, we identify new areas for future visualization research. As one example, more research is needed on how to strategically design example galleries to boost user creativity and innovation, which would benefit not only D3 users but users of all visualization languages.

2 RELATED WORK

Analyzing Online Forums. A number of projects investigate the practices [26, 44, 46], attitudes [11, 22] and impacts [20, 34] of engagement in various online Q&A communities, and the impact online forums can have on individual users [3, 4, 14, 28, 43]. We utilize prior work to explore the intersection between online Q&A communities and visualization implementation workflows and behaviors. We observe similar phenomena reported in prior work, such as a large fraction of unanswered questions [43], strong associations between certain tools and libraries [13], prevalent code re-use [2], and an emphasis on web development [5]. However, given our focus on visualization, we can speak to impacts on D3 users specifically, and adapt existing guidance to the interests and needs of the visualization community. We believe this paper is the *first* to analyze D3 implementation workflows by mining posts from online forums.

Visualization Mining, Modeling, and Analysis. Several projects highlight the value in analyzing existing designs from the web, such as website designs [23], visualization designs [1, 6, 16, 19, 36], and even code structure [17]. Online platforms such as bl.ocks.org [9] and Observable [30] also help communities engage with visualizations online. A number of works analyze visualization images to extract authors' design decisions (e.g., [6, 16, 21, 33, 40]). We extend prior work by automating the process of analyzing users' challenges in implementing visualizations, rather than just analyzing the visualizations themselves. Our findings could guide the design of automated techniques, e.g., by extending systems such as VizML [19] and Draco [29], to identify and fill specific blind spots in a user's visualization knowledge in a way that aligns with existing user implementation workflows.

3 ANALYSIS OVERVIEW

Our primary objective in this work is to identify opportunities to enhance the D3 visualization implementation process. To this end, we use the original D3 design goals to drive our analysis (compatibility, debugging, performance) [10]. For sake of space, we focus

on two of these goals in this paper: *compatibility* and *debugging*¹. We summarize each goal as a concrete question for our analysis:

1. **Compatibility:** How is D3 used in conjunction with other tools and environments?
2. **Debugging:** How do users explore, interpret, and augment the behavior of D3 code?

To answer these questions, we downloaded relevant Stack Overflow posts, using Stack Overflow's tag search to limit the corpus to posts that include the "d3.js" tag. Only posts that were accessible on Stack Overflow were included. With our approach, we are able to analyze **37,815 total StackOverflow posts** from 17,591 unique D3 users, showing the power of scale afforded by our techniques.

We apply a mixed methods analysis approach:

1. **Explore** a randomized sample of posts to identify patterns of potential interest as qualitative codes;
2. **Filter** the full 37,815 corpus using specific keyphrases derived from our qualitative codes and associated quotes;
3. **Count** observations of keyphrases on the full corpus to produce quantitative measures;
4. **Compare** our results to follow-up analyses of Stack Overflow posts, the D3 documentation, or relevant GitHub issues and release notes (if applicable), to provide additional context.

In line with prior qualitative studies (e.g., [20, 26]), we created a representative sample of Stack Overflow posts for the **explore** phase of our analysis pipeline. This sample contains 817 posts randomly sampled through the year 2020. Three authors manually coded these posts using *descriptive* deductive and inductive codes. Deductive codes were used for known categories in visualization, such as visualization types (e.g., [6]) and interaction types (e.g., [12]). Inductive codes labeled self-reported visualization and/or user characteristics, such as labeling when bar charts, Adobe Illustrator, or SQL were mentioned. Codes were not exclusive, and could be applied in parallel. Note that for most of our qualitative findings from the **explore** phase, we verify our results quantitatively using the full corpus (in Sect. 4.1, Sect. 5.1, and Sect. 5.2).

4 COMPATIBILITY: INTEGRATING D3 WITH OTHER TOOLS

In this section, we analyze mentions of external APIs and tools in our random sample, and summarize the types of issues that Stack Overflow users discuss online.

4.1 Analyzing Users' Visualization Toolsets

23.4% of all posts we qualitatively analyzed referenced at least one other external library or tool. We found 55 different languages, tools, and libraries mentioned in conjunction with D3.js. To see if these frequencies suggest a larger pattern, we compared with observations of the same tools in the full Stack Overflow corpus. We found that the random sample and full corpus have very similar distributions, pointing to broader patterns in how D3 is being used in conjunction with other tools (please see our technical report for more details). We highlight a few interesting observations here.

D3 is Often Integrated Into Larger Web Applications. For example, D3 users often use React, Angular, or Vue to manage the front-end, and Python, Node, Ruby on Rails, Electron, or Java Spring to run the back-end. 34 of 55 (or 61.8%) of the tools we observed are JavaScript Libraries, and React.js, Angular, dc.js, jQuery, and NVD3 are mentioned the most. However, each tool covers a small fraction of posts. For example, React is mentioned in 5% of coded posts. Thus there are a few popular application structures, but Stack Overflow users vary widely in the libraries/tools they use with D3.

D3 is Paired With Specialized Visualization Libraries. Though D3 provides support for geographic maps and graphs, we find some users attempting to integrate D3 with specialized visualization libraries, such as leaflet, simplify, datamaps, and openweathermap for maps, and web cola for graphs. Greensock and Three.js, specialized libraries for 2D and 3D animation, were also mentioned.

¹Please see our technical report for more details: <https://arxiv.org/abs/2108.02299>.

D3 is Used Outside of JavaScript. D3 is not used solely with JavaScript. For example, some Stack Overflow users seek help using D3 in R and Jupyter Notebooks. Users mentioned three packages in particular for R: shiny, r2d3, and radialnetworkr. Users also mention other computational environments, such as PostgreSQL and SparQL. However, the overwhelming majority of Stack Overflow posts that we analyzed focused on a JavaScript programming context.

Graphics Editors and Spreadsheets are Used for Prototyping. We also find some interesting implementation variants, such as using Microsoft Excel or Adobe Illustrator for brainstorming prior to implementation (e.g., posts D-73 and A-296 from our dataset files, respectively). Three graphics editors were mentioned on Stack Overflow (Illustrator, CorelDraw, and InkScape), suggesting that a number of users brainstorm visualizations in non-code environments as part of their implementation workflows, and prior to using D3.

4.2 Common Assumptions Clash With User Workflows

When investigating one of the more popular application structures (React components), we find two common integration challenges. First, React is known to have a steep learning curve, and Stack Overflow users often encounter challenges simply in getting React to work properly. For example, we find that many posters have more trouble understanding React than D3 itself (e.g., post B-502). The second theme we observe is a clash in functionality between React and D3. D3 was originally designed to manipulate the DOM directly [10], with known issues for integrating D3 with other libraries that also modify the DOM such as React [41]. We observed this integration challenge in our qualitative analysis; for example, one poster's solution to their D3 integration problem was to use React-focused utilities designed to integrate with D3, rather than use D3 directly (post B_C-63). We also observed Stack Overflow users mentioning libraries that replace D3's DOM manipulation operations with those of another library, such as how ngx-charts uses Angular instead of D3 for rendering purposes (e.g., post D-38). Some answers even suggested removing D3 entirely (e.g., post B-149).

We observe that when incorporating a JavaScript library that manipulates the DOM, a Stack Overflow user tends to anchor their workflow on this library, since the DOM is the physical structure of the webpage itself. By choosing to manipulate the DOM directly, earlier versions of D3 tried to act as the anchoring library. Thus in a way, the design of D3 assumed that D3 was the focal point of an implementation workflow, even though D3 is scoped primarily for data interaction and visualization, which represent only a fraction of a user's overall interface and webpage design. However, in reviewing D3's release notes, we found that D3's structure has shifted significantly over time from being an anchoring language to a modular collection of data manipulation and visualization libraries [8].

D3's evolution in response to user (and developer) implementation challenges provides critical context for how visualization tools can be designed in the future. For example, the visualization community often develops and evaluates new tools without considering how they will be incorporated into existing workflows [31, 42]. This lack of awareness could lead to less functional tools, and ultimately lower adoption and impact for visualization work.

4.3 Takeaways.

We see a wide range of libraries and tools used as part of larger visualization implementation workflows, from specialized visualization libraries to usage with non-JavaScript languages such as Python and R, and even graphics editors and spreadsheets. We explore how D3 evolved in response to users' implementation challenges, which may speak to broader limitations in how visualization research is conducted, in particular the perspective that visualization tools are the focal point of a user's visualization implementation workflow. Our findings suggest that the visualization community may benefit from taking the goal of compatibility even further, for example by treating visualizations as just one component of a larger application that a user wants to create, rather than the main focus. This shift

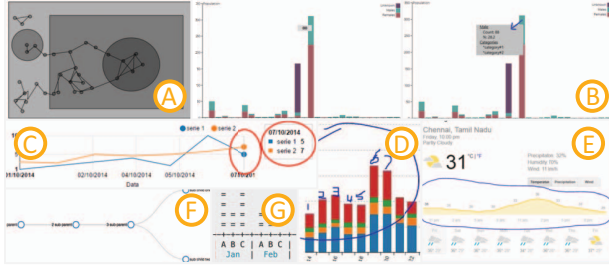


Figure 1: Example images shared online to convey desired visualizations (A, E, F, & G), interactions (B), and modifications (C & D).

in perspective necessitates a change in how visualization tools are designed and evaluated. Long term case studies could be a useful starting point for this shift [42].

5 DEBUGGING: INTERPRETING & APPLYING D3 CONCEPTS In this section, we qualitatively analyze the kinds of bugs that Stack Overflow users often run into with D3, and the different strategies they use to articulate and fix their D3 implementation bugs.

5.1 Analyzing Implementation & Debugging Methods

Stack Overflow Users Wrestle With Odd D3 Behavior More Often than Explicit Errors. Users mention explicit compilation or runtime errors only 10.6% of the time (87 out of 817 posts). Instead, Stack Overflow users' bugs tend to involve runnable code that exhibits unexpected or unwanted behaviors (337 out of 817 posts, or 41.2%), especially unexpected rendering effects or unexpected interaction behaviors in the visualization output.

Stack Overflow users often rely on existing examples to find solutions to their bugs. We find that Stack Overflow users often reference existing D3 examples that are publicly available online when discussing their bugs (179 out of 817, or 21.9% of posts). This suggests that Stack Overflow users often rely on existing examples when debugging their D3 code. To better understand the influence of the D3 documentation on example usage, we counted the number of Stack Overflow posts containing a link to examples from the visual index of the D3 visualization gallery, bl.ocks.org, and Observable [30]. We find that 13.6% of all posts directly reference examples from just these three sources, representing a significant fraction of all referenced examples in our qualitative dataset.

To see if these findings point to a larger pattern, we compare them with the full Stack Overflow corpus. Across all 37,815 posts analyzed, we find that 14% include references to bl.ocks.org, Observable, or the D3 Gallery visual index, which is consistent with our coded data. Thus D3 examples and documentation seem to be key components of users' D3 visualization implementation workflows.

5.2 Challenges in Using Relevant Examples to Fix Bugs Adapting existing D3 examples seems to be an important but also complicated part of the D3 implementation and debugging process, which we explore further in this section.

Some Users Struggle to Find the Most Relevant Examples. When searching for relevant examples, D3 users may struggle to match their own terminology for different visualization and interaction types to the terminology of others. For example, we find that 46 of 817 (or 5.6%) of posts mention difficulties in finding *any* relevant D3 examples or Stack Overflow posts. If a D3 user is unaware of the appropriate D3 methods for implementing certain functionality, then a typical keyword search in Google or Observable may fail due to mismatches in terminology. Advanced visualization search engines may also be of limited use, due to their reliance on language-specific terminology, such as [18] and bl.ocksplorer.org. However, once relevant examples are identified, D3 users still seem to struggle with separating relevant code from irrelevant code, and with distinguishing D3 components from those of other libraries.

Complicated, Irrelevant Functionality Makes Examples Hard to Reuse. The utility of a D3 example seems to depend on not only the ratio of relevant to irrelevant functionality, but also the complexity of this functionality. Unfortunately, existing examples routinely contain more functionality than the user wants or is familiar with, which can easily confuse them and lead to unnecessary bugs. Consider the following quote, where a user is struggling to reuse the zoom functionality from a geographic map example with meshes:

"I am trying to implement a 'zoom to bound' feature on my D3 map. This is the Block that I have in mind... My issue is that it looks like the implementation requires a topoJSON mesh." (post A-587)

From follow-up comments to the post, we learn that meshes are not needed, revealing unnecessary functionality:

"You don't need the mesh, that's just for the states' strokes." (post A-587)

The commentor also suggests a more relevant example *"Have a look at this bl.ocks without mesh..."* (post A-587), pointing to a separate problem that we observed: some users struggle to find the most relevant examples in the first place.

Users Struggle to Connect D3 Code Components With Their Corresponding Visual Outputs. We find that many Stack Overflow users struggle to pinpoint the root causes of their D3 bugs. One challenge for Stack Overflow users may be that the way in which users intuitively reason about visualization outputs may not match how these outputs are generally specified using the D3 language. Rather than using the parlance of the D3 language to describe what they want to create, many Stack Overflow users illustrate their desired outputs by embedding one or more images within a post, often with annotations added. For example, 21.1% of the posts we analyzed included linked or embedded images, which also held true for our full Stack Overflow corpus (19.2%).

The annotations generally appeared to be hand-drawn, or applied using image editing or presentation software. Fig. 1 provides examples from five different Stack Overflow posts. For example in Fig. 1-E, the user circled a specific visualization within an existing image (online weather information) that they want to copy. In Fig. 1-G, the user has made a diagram in ASCII showing how they want to order and group the bars of a bar chart, reminiscent of ASCII-based diagramming seen in other debugging contexts (e.g., database debugging [7]). Some users would even go so far as to draw the specific changes they expect to make. In Fig. 1-D, the user has manually illustrated their desired results by annotating a stacked bar chart with the totals drawn above each stack. Users also illustrate desired interactions by sharing videos (e.g., post E-110), .gifs (e.g., post D-38), and sequences of annotated images (e.g., Fig. 1-A and B).

5.3 Takeaways.

Stack Overflow users often post about unexpected behaviors rather than explicit errors. One common implementation and debugging strategy among these users is to compare their code with relevant examples that are publicly available online. However, Stack Overflow users often struggle to find the most relevant D3 examples to inform their coding and debugging process. When they do find relevant examples, they may still struggle to extract the most relevant functionality. This is due in part to how many D3 examples lack a modular structure, and often contain functionality that other users do not need, which can lead to confusion and unnecessary bugs.

If users could search for desired functionality without domain-specific keywords, they could find relevant D3 examples with less time and effort. Furthermore, some users have found their own way of communicating desired visualization outputs through annotated images, videos, and gifs. If Stack Overflow users could demonstrate their desired changes by *directly manipulating the visualization output* (e.g., [25, 35, 47]), then they could debug their code using a search-by-example process, rather than a language or keyword-based

search. Then, visualization tools could compute the corresponding implementation deltas and update the code automatically. Debugging via direct manipulation has been proposed in other contexts, such as for updating Jupyter notebooks [45] and debugging SQL queries [15], and could be adapted for visualization languages.

6 DISCUSSION: IMPLICATIONS FOR FUTURE RESEARCH

D3 has made incredible contributions to the visualization community. In this paper, we investigate opportunities to further enhance the experience of D3 users and of visualization language users in general. We present an analysis of 37,815 posts made by D3 users on Stack Overflow. We evaluate D3 from two perspectives: compatibility and debugging. Our findings show that when we focus on developing visualization languages but not how and where people actually use them, we may struggle to fully understand the user experience, and may thus fail to fully identify and address the needs of these users. By being mindful of how users interact with visualization languages and relay their implementation challenges, we can develop innovative strategies to enhance users' implementation processes, increase users' information access, and empower users to explore a wider range of effective visualization designs. In this section, we highlight opportunities for future work based on our findings.

6.1 Emphasize Integration With Non-Visualization Tools

When visualizations become the sole focus of visualization tool development, developers and researchers may build tools that conflict with other critical needs within users' implementation workflows, potentially hindering adoption. As demonstrated through the evolution of D3, our community needs to shift its mindset towards building modular visualization *components* that can integrate smoothly with other tools. Furthermore, we encourage more formal evaluations of how users integrate new tools and languages (or not) into their implementation workflows over time [31, 42]. For example, we encourage our community to conduct more large scale, quantitative studies of how visualization languages are used in popular development environments, e.g., Jupyter, Observable, and R Studio.

6.2 More Modular Infrastructure Could Boost Adoption

We find that users on Stack Overflow often rely on examples to implement and debug new D3 visualizations, but struggle to find relevant examples and reuse them correctly. Our findings point to two challenges in supporting current debugging workflows. First, we lack *modularized building blocks* for implementing new visualizations in D3. This issue may stem in part from D3's mixing of declarative specification of encodings with imperative specification of interactions, which is addressed in later languages such as Vega-Lite [38]. Second, the problem may not only be with code structure but also *insufficient infrastructure* for helping users understand the flow of the code, i.e., how results propagate through the various parts of a D3 visualization. This issue has led to recent developments such as the Observable notebook environment [30], but Observable still expects users to manually segment their own code. Similar environments such as Jupyter notebooks suffer from the same problems. Both challenges highlight how D3 users struggle to break their implementation challenges down into modular, solvable pieces.

Based on our findings, we argue that both challenges could be addressed effectively by improving the *support infrastructure* around D3, rather than by modifying D3 itself. For example, we could develop more intuitive search interfaces that support search by example or search by demonstration, such as searching existing D3 examples for specific visual outputs or intended interaction behaviors. Solutions can be developed in a data-driven way by mining them from the thousands of existing D3 examples we observed. For example, new tools could leverage this data to help users identify separate D3 components within existing examples, and extract only those that are needed. Development environments could also be augmented to automatically recommend relevant documentation or code blocks based on inferred user goals, experience levels, and potential biases.

6.3 Make Example Gallery Design Active Research

Our findings in Sect. 5 show that a basic web search is just not good enough to help users find relevant D3 examples. Asking users to search for solutions using specialized D3 or visualization keywords are sub-optimal alternatives. Even Stack Overflow is insufficient; translating a specific D3 bug into a self-contained Stack Overflow question requires significant time and effort [14], and 37% of the 37,815 posts analyzed were left unanswered by other users. We believe these issues persist because our community views them as engineering rather than research problems.

In an effort to shift this perspective, we highlight opportunities to expand existing design galleries. We could synthesize current best practices in visualization design as a diverse set of modularized visualization examples that all visualization language developers aim to provide. Rather than expecting developers to create design galleries meeting these requirements, we could also find ways to *automate* the generation of design galleries. This solution could involve a mixture of automation and the crowd, where automated processes are developed not only to detect gaps in existing galleries, but also to encourage users to fill these gaps with new examples. This approach could also be used to detect redundant or low quality examples and replace them automatically. Developing design galleries and documentation takes time, whether for open-source languages such as Vega-Lite [38], or commercial APIs such as Plotly [32]. Automating the documentation process could speed up the learning process and dampen learning curves for users of *all* visualization languages.

6.4 Takeaways Summary

Here we list three major takeaways derived from our research:

- Develop and test visualization languages as part of larger implementation workflows involving multiple tools.
- Provide support infrastructure for helping users find relevant examples, extract meaningful code components, and integrate these components into their workflows.
- Automate the example gallery generation process, to ease the difficulty of designing effective examples and make the process more consistent across languages/tools.

6.5 Limitations and Future Work

One limitation is that we only focus on D3 users who post on Stack Overflow, a subset of all D3 users. However, we are still able to study 17,591 total D3 users, showing the scale afforded by our approach. Given that posters do not have to share personal information on Stack Overflow, user characteristics are not consistently available in our dataset; thus we exclude them from our analysis. An interesting direction for future research is to conduct follow up interviews with D3 users to better understand their backgrounds, motivations, and experiences, providing additional context for our findings. However, we believe that our ability to analyze thousands of D3 users helps to balance this limitation out. Certain D3 functionality may not be well represented in our dataset, e.g., animations. It would be interesting to introduce filters to our Stack Overflow crawler to extract specific posts for more targeted analyses in the future, e.g., downloading animation-focused posts for further analysis. In general, we hope that by sharing our materials, we can empower the community to explore visualization languages in new ways. For example, a promising avenue of future work could be to analyze iteration on visualization languages and user reasoning in tandem over time.

ACKNOWLEDGMENTS

This research was funded in part by NSF Award #1850115. We thank the UMD HCIL, the BAD Lab, the UW IDL, and our (many) reviewers for their invaluable feedback. We also thank Arjun Nair and Rishik Narayana for their help with data collection.

REFERENCES

- [1] End-users Publishing Structured Information on the Web: An Observational Study of What, Why, and How. CHI '14. doi: 10.1145/2556288.2557036

- [2] R. Abdalkareem, E. Shihab, and J. Rilling. On code reuse from stackoverflow: An exploratory study on android apps. *Information and Software Technology*, 88:148–158, 2017.
- [3] A. Anderson, D. Huttenlocher, J. Kleinberg, and J. Leskovec. Discovering Value from Community Activity on Focused Question Answering Sites: A Case Study of Stack Overflow. *KDD '12*, pp. 850–858. ACM, 2012. doi: 10.1145/2339530.2339665
- [4] A. Ardichvili, M. Maurer, W. Li, T. Wentling, and R. Stuedemann. Cultural influences on knowledge sharing through online communities of practice. *Journal of Knowledge Management*, 10(1):94–107, Jan. 2006. doi: 10.1108/13673270610650139
- [5] A. Barua, S. W. Thomas, and A. E. Hassan. What are developers talking about? an analysis of topics and trends in stack overflow. *Empirical Software Engineering*, 19(3):619–654, 2014.
- [6] L. Battle, P. Duan, Z. Miranda, D. Mukusheva, R. Chang, and M. Stonebraker. Beagle: Automated Extraction and Interpretation of Visualizations from the Web. *CHI '18*, pp. 594:1–594:8. ACM, 2018. doi: 10.1145/3173574.3174168
- [7] L. Battle, D. Fisher, R. DeLine, M. Barnett, B. Chandramouli, and J. Goldstein. Making Sense of Temporal Queries with Interactive Visualization. *CHI '16*, pp. 5433–5443. ACM, 2016. doi: 10.1145/2858036.2858408
- [8] M. Bostock. Release v4.0.0 - d3/d3, 2016.
- [9] M. Bostock. Popular blocks - blocks.org, 2021.
- [10] M. Bostock, V. Ogievetsky, and J. Heer. D3 Data-Driven Documents. *IEEE TVCG*, 17(12):2301–2309, Dec. 2011. doi: 10.1109/TVCG.2011.185
- [11] A. Bosu, C. S. Corley, D. Heaton, D. Chatterji, J. C. Carver, and N. A. Kraft. Building Reputation in StackOverflow: An Empirical Investigation. *MSR '13*, pp. 89–92. IEEE Press, 2013.
- [12] M. Brehmer and T. Munzner. A multi-level typology of abstract visualization tasks. *IEEE TVCG*, 19(12):2376–2385, 2013. doi: 10.1109/TVCG.2013.124
- [13] C. Chen and Z. Xing. Mining technology landscape from stack overflow. *ESEM '16*. ACM, 2016. doi: 10.1145/2961111.2962588
- [14] D. Ford, J. Smith, P. J. Guo, and C. Parnin. Paradise Unplugged: Identifying Barriers for Female Participation on Stack Overflow. *FSE 2016*, pp. 846–857. ACM, 2016. doi: 10.1145/2950290.2950331
- [15] S. Gathani, P. Lim, and L. Battle. Debugging database queries: A survey of tools, techniques, and users. *CHI '20*, p. 1–16. ACM, New York, NY, USA, 2020.
- [16] J. Harper and M. Agrawala. Converting Basic D3 Charts into Reusable Style Templates. *IEEE TVCG*, 24(3):1274–1286, Mar. 2018. doi: 10.1109/TVCG.2017.2659744
- [17] A. Head, E. L. Glassman, B. Hartmann, and M. A. Hearst. Interactive Extraction of Examples from Existing Code. *CHI '18*, pp. 85:1–85:12. ACM, 2018. doi: 10.1145/3173574.3173659
- [18] E. Hoque and M. Agrawala. Searching the visual style and structure of d3 visualizations. *IEEE TVCG*, 26(1):1236–1245, Jan 2020. doi: 10.1109/TVCG.2019.2934431
- [19] K. Hu, M. A. Bakker, S. Li, T. Kraska, and C. Hidalgo. VizML: A Machine Learning Approach to Visualization Recommendation. *CHI '19*, pp. 128:1–128:12. ACM, 2019. doi: 10.1145/3290605.3300358
- [20] R. Jones, L. Colusso, K. Reinecke, and G. Hsieh. R/science: Challenges and opportunities in online science communication. *CHI 2019*, p. 1–14. ACM, New York, NY, USA, 2019.
- [21] D. Jung, W. Kim, H. Song, J.-i. Hwang, B. Lee, B. Kim, and J. Seo. ChartSense: Interactive Data Extraction from Chart Images. *CHI '17*, pp. 6706–6717. ACM, 2017. doi: 10.1145/3025453.3025957
- [22] T. Kauer, M. Dörk, A. L. Ridley, and B. Bach. The public life of data: Investigating reactions to visualizations on reddit. *CHI '21*. ACM, New York, NY, USA, 2021.
- [23] R. Kumar, A. Satyanarayan, C. Torres, M. Lim, S. Ahmad, S. R. Klemmer, and J. O. Talton. Webzeitgeist: Design Mining the Web. *CHI '13*, pp. 3083–3092. ACM, 2013. doi: 10.1145/2470654.2466420
- [24] H. Lam, E. Bertini, P. Isenberg, C. Plaisant, and S. Carpendale. Empirical studies in information visualization: Seven scenarios. *IEEE TVCG*, 18(9):1520–1536, 2012. doi: 10.1109/TVCG.2011.279
- [25] B. Lee, G. Smith, N. H. Riche, A. Karlson, and S. Carpendale. Sketchinsight: Natural data exploration on interactive whiteboards leveraging pen and touch interaction. *PacificVis '15*, pp. 199–206, 2015. doi: 10.1109/PACIFICVIS.2015.7156378
- [26] K. Mack, J. Lee, K. Chang, K. Karahalios, and A. Parameswaran. Characterizing scalability issues in spreadsheet software using online forums. *CHI EA '18*, p. 1–9. ACM, New York, NY, USA, 2018. doi: 10.1145/3170427.3174359
- [27] H. Mei, Y. Ma, Y. Wei, and W. Chen. The design space of construction tools for information visualization: A survey. *Journal of Visual Languages & Computing*, 44:120–132, Feb. 2018. doi: 10.1016/j.jvlc.2017.10.001
- [28] A. Merchant, D. Shah, G. S. Bhatia, A. Ghosh, and P. Kumaraguru. Signals Matter: Understanding Popularity and Impact of Users on Stack Overflow. *WWW '19*, pp. 3086–3092. ACM, 2019. doi: 10.1145/3308558.3313583
- [29] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in draco. *IEEE TVCG*, 25(1):438–448, 2019. doi: 10.1109/TVCG.2018.2865240
- [30] Observable, Inc. Observable - make sense of the world with data, together / observable, 2021.
- [31] C. Plaisant. The challenge of information visualization evaluation. *AVI '04*, p. 109–116. ACM, 2004. doi: 10.1145/989863.989880
- [32] Plotly. Plotly: The front end for ml and data science models, 2021.
- [33] J. Poco and J. Heer. Reverse-Engineering Visualizations: Recovering Visual Encodings from Chart Images. *Comput. Graph. Forum*, 36(3):353–363, June 2017. doi: 10.1111/cgf.13193
- [34] S. Ravi, B. Pang, V. Rastogi, and R. Kumar. Great question! question quality in community q&a. 8(1):426–435, May 2014.
- [35] B. Saket, A. Srinivasan, E. D. Ragan, and A. Endert. Evaluating Interactive Graphical Encodings for Data Visualization. *IEEE TVCG*, 24(3):1316–1330, Mar. 2018. doi: 10.1109/TVCG.2017.2680452
- [36] B. Saleh, M. Dontcheva, A. Hertzmann, and Z. Liu. Learning Style Similarity for Searching Infographics. *GI '15*, pp. 59–64. Canadian Information Processing Society, 2015.
- [37] A. Satyanarayan, B. Lee, D. Ren, J. Heer, J. Stasko, J. Thompson, M. Brehmer, and Z. Liu. Critical reflections on visualization authoring systems. *IEEE TVCG*, 26(1):461–471, 2020. doi: 10.1109/TVCG.2019.2934281
- [38] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-Lite: A Grammar of Interactive Graphics. *IEEE TVCG*, 23(1):341–350, Jan. 2017. doi: 10.1109/TVCG.2016.2599030
- [39] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive Vega: A Streaming Dataflow Architecture for Declarative Interactive Visualization. *IEEE TVCG*, 22(1):659–668, Jan. 2016. doi: 10.1109/TVCG.2015.2467091
- [40] M. Savva, N. Kong, A. Chhajta, L. Fei-Fei, M. Agrawala, and J. Heer. ReVision: Automated Classification, Analysis and Redesign of Chart Images. *UIST '11*, pp. 393–402. ACM, 2011. doi: 10.1145/2047196.2047247
- [41] D. Scanlon. How (and why) to use d3 with react — hacker noon, 2017.
- [42] B. Shneiderman and C. Plaisant. Strategies for evaluating information visualization tools: Multi-dimensional in-depth long-term case studies. *BELIV '06*, p. 1–7. ACM, 2006. doi: 10.1145/1168149.1168158
- [43] R. Slag, M. de Waard, and A. Bacchelli. One-day flies on stackoverflow - why the vast majority of stackoverflow users only posts once. *MSR '15*, pp. 458–461. IEEE, 2015. doi: 10.1109/MSR.2015.63
- [44] C. Treude, O. Barzilay, and M.-A. Storey. How Do Programmers Ask and Answer Questions on the Web? (NIER Track). *ICSE '11*, pp. 804–807. ACM, 2011. doi: 10.1145/1985793.1985907
- [45] Y. Wu, J. M. Hellerstein, and A. Satyanarayan. *B2: Bridging Code and Interactive Visualization in Computational Notebooks*, p. 152–165. *UIST 2020*. ACM, 2020.
- [46] T. Zhang, G. Upadhyaya, A. Reinhardt, H. Rajan, and M. Kim. Are code examples on an online q&a forum reliable? a study of api misuse on stack overflow. *ICSE '18*, p. 886–896. ACM, 2018. doi: 10.1145/3180155.3180260
- [47] J. Zong, D. Barnwal, R. Neogy, and A. Satyanarayan. Lyra 2: Designing interactive visualizations by demonstration. *IEEE TVCG*, 27(2):304–314, 2021. doi: 10.1109/TVCG.2020.3030367